File I/O 

Christopher Martin - *Bowdoin College*
c.martin@bowdoin.edu

Today, we are going to review how we can have our programs interact with files on our computers

This is very important and expands on the capabilities of our programs in a way similar to taking user input...

Allowing our programs to read and write data from external files greatly expands the ways our programs can use and produce data!

Opening a File

The first step involved with trying to read from or write to a file is opening it so our program has access to its data as a variable!
We can accomplish this using the `open` function!

The syntax template for the `open` function is as follows:

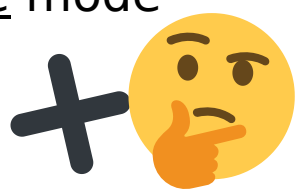
```
1 with open(FILE_NAME, MODE, encoding="utf-8") as file:
2   # Do something with the 'file' variable
```

- `FILE_NAME` **can be the name or** path **of a file or folder (called a directory)**
  `open` assumes files exist inside the same directory as the folder you have open in VSCode
- `MODE` **controls the operations we are capable of performing on the file**
  We will review the specific values you can provide here shortly!

File Modes

Mode	Description	Caveats
`"r"`	Opens the file in read-only* mode from the beginning	Fails if the file does not exist
`"w"`	Opens the file in write-only* mode from the beginning	Creates a new file if it does not exist or overwrites the file if it exists
`"a"`	Opens the file in write-only* mode from the end	Creates a new file if it does not exist

You can add a `+` to any mode to open the file in read and write mode
This does not mean `"r+"`, `"w+"`, and `"a+"` are the same!



FileNotFoundError

You will get an error if you try to open something that can't be found
Remember that `open` assumes the file / directory is within the current folder

It is possible to get this error if the file exists somewhere else or not at all!

```
1 with open("example.txt", "r", encoding="utf-8") as file:
2   ...
```

```
Traceback (most recent call last):
  File "/app/output.s", line 1, in <module>
    with open("example.txt", "r", encoding="utf-8") as file:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'example.txt'
```

File Input

The following methods can be used to extract data from a file

These methods will fail if the file is not opened in read mode!

Method	Description
`file.read()`	Reads and returns all remaining characters in `file` as a string
`file.read(n)`	Reads and returns the next `n` characters of `file` as a string
`file.readline()`	Reads and returns all remaining characters in `file` until a newline is encountered as a string
`file.readlines()`	Reads and returns all remaining characters in `file` broken up by line as a list of strings

File Output

The following methods can be used to write data to a file

These methods will fail if the file is not opened in write mode!

Method	Description
`file.write(s)`	Writes the given string `s` to `file` at the current cursor position
`file.writelines(l)`	Writes each of the strings in the given list `l` to `file` from the current cursor position

File Cursor

The following methods can be used to manipulate the file cursor!

Method	Description
`file.tell()`	Returns the current file cursor position
`file.seek(p)`	Changes the current file position to the character position `p`
`file.seek(w, p)`	Changes the current file position to the character position `p` relative to `w`

`w` can have one of three values:

- `0` -> From the beginning of the file
- `1` -> Relative to the current cursor position
- `2` -> From the end of the file

Why is this important

Similarly to user input, file input allows our programs to operate dynamically based on the data they are reading!
This is especially good if we are dealing with a lot of data!

File output allows us to save data when our program finish!
Normally, all the state in our programs go away when it finishes executing!

Saving state can be really useful, especially when we want data from a previous occurrence of the program to be used by future executions!

What are some examples of data that is saved between program executions?